

WEEK 1 MODULE

Part 1: Handwritten Notes

Week 1 Module

1. Syntax

↳ Definition: Syntax refers to the set of grammatical rules that ~~dictate~~ dictate how a C program is structured and written.

- ↳ Key Rules:
- i) Every statement in C must terminate with a semicolon (;)
 - ii) Blocks of code are grouped using curly braces {}
 - iii) C is a case-sensitive language
 - iv) Comments are written using // for single lines or /*....*/ for multiple lines.

2. Condition and Decision Making

↳ Definition: Conditional statement allows a program to evaluate boolean expressions (true/false) and execute specific blocks of code based on the outcome.

- ↳ Types:
- i) if statement - Executes a block of code only if a specified condition evaluates to true.
 - ii) else statement - Follows an if block and executes if the if condition evaluates to false.
 - iii) else if statement - Used to chain multiple conditions together to be checked in sequence.
 - iv) Switch statement - Evaluates a single variable and executes a matching block, which is useful as an alternative to long else if chains.

3. Loops.

↳ Definition: Loops are control structures that repeatedly execute a block of code as long as a specified condition remains true. They are heavily used in tasks like printing patterns or ~~traversing~~ traversing arrays.

↳ Types: i) for loop: Ideal when the exact no. of iteration is known in advance. It includes initialization, condition-checking, and increment/decrement in a single line.

• Syntax:- `for (initialization; condition; update)
{ //code }`

ii) while loop: Continues to execute as long as its condition is true. The condition is checked before the code runs.

iii) do-while loop: Similar to the while loop, but the condition is checked after the code runs, guaranteeing that the code executes at least once.

Part 2: C Programming Problem Codes

1. Square Pattern

```
#include <stdio.h>

int main() {
    int n = 5;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
*****
*****
*****
*****
*****
```

=== Code Execution Successful ===

2. Hollow Square Pattern

```
#include <stdio.h>

int main() {
    int n = 5;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(i == 0 || i == n-1 || j == 0 || j == n-1) {
                printf("*");
            } else {
                printf(" ");
            }
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
*****
*      *
*      *
*      *
*****
```

=== Code Execution Successful ===

3. Triangle Pattern

```
#include <stdio.h>

int main() {
    int n = 5;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
*
**
***
****
*****
```

=== Code Execution Successful ===

4. Inverted Triangle Pattern

```
#include <stdio.h>

int main() {
    int n = 5;
    for(int i = n; i >= 1; i--) {
        for(int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Output

**

*

=== Code Execution Successful ===

5. Pyramid Pattern

```
#include <stdio.h>

int main() {
    int n = 5;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for(int k = 1; k <= (2 * i - 1); k++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
  *
 ***
*****
*****
*****
```

=== Code Execution Successful ===

6. Inverted Pyramid Pattern

```
#include <stdio.h>

int main() {
    int n = 5;
    for(int i = n; i >= 1; i--) {
        for(int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for(int k = 1; k <= (2 * i - 1); k++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Output

*

=== Code Execution Successful ===

7. Half Diamond Pattern

```
#include <stdio.h>

int main() {
    int n = 5;

    // Top half
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    // Bottom half
    for(int i = n - 1; i >= 1; i--) {
        for(int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
*
**
***
****
*****
****
***
**
*
```

=== Code Execution Successful ===

8. Diamond Pattern

```
#include <stdio.h>

int main() {
    int n = 4; // Top half has 4 rows to match the reference shape

    // Top half
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n - i; j++) printf(" ");
        for(int k = 1; k <= (2 * i - 1); k++) printf("*");
        printf("\n");
    }

    // Bottom half
    for(int i = n - 1; i >= 1; i--) {
        for(int j = 1; j <= n - i; j++) printf(" ");
        for(int k = 1; k <= (2 * i - 1); k++) printf("*");
        printf("\n");
    }

    return 0;
}
```

Output

```
  *
 ***
*****
*****
*****
  ***
  *
```

=== Code Execution Successful ===

****THE END****